



VEHICULOS

Qué le vence a tu moto o a tu carro, y cuándo.

Programación Móvil (IF2004) · Grupo 602 · Entregable 1: definición y navegación

David Muñoz Betancur · Juan Pablo Bustos Sepúlveda · Anderson Fabián Sánchez Benítez

26 de septiembre de 2026

El problema: el mantenimiento se lleva de memoria



Se pasan los kilómetros

El desgaste se mide en tiempo ("hace como tres meses") y no en km, que es la medida real. Una moto de uso diario pasa los 3.000 km del aceite en seis semanas.



Se pasan las fechas

SOAT, técnico-mecánica e impuesto vencen una vez al año, en fechas distintas, y nadie avisa. Circular sin ellos: multa e inmovilización (Ley 769 de 2002, art. 131).



El mecánico es la única fuente

Depende de lo que el conductor le cuente y tiene interés en que el vehículo vuelva al taller.



Caso real del equipo

Suzuki Gixxer 250

El manual pide el cambio de aceite cada 3.000 km o 3 meses, lo que ocurra primero. Como es una moto secundaria, la cuenta se llevaba por meses y de memoria. El cambio se hizo después del intervalo.

Con la app, ese aceite hoy aparecería como "vencido por tiempo".

¿Por qué una app móvil y no una hoja de cálculo?



El dato se lee en el tablero

El kilometraje se ve con el vehículo al frente y el teléfono en la mano, no frente a un computador.



La alerta se necesita antes de salir

Saber si hoy hay pico y placa o si el SOAT vence mañana sirve en la puerta, no al final del día.



Funciona sin conexión

En un parqueadero o en carretera no siempre hay señal. Todo se guarda en el teléfono.

Objetivos



Objetivo general

Que el conductor sepa, desde su teléfono y sin depender del taller, qué mantenimiento y qué documento le vence a cada vehículo y en cuánto tiempo.

15 días

de anticipación mínima con la que cada vencimiento aparece en el panel.

6 toques

como máximo para registrar un vehículo después del inicio de sesión.

2 toques

para actualizar el kilometraje desde el panel del vehículo.

Al abrir

el panel se ve si hoy hay pico y placa, sin buscarlo en otra parte.

Cada objetivo tiene un número: así se sabe al final si se logró.

Alcance de esta versión



Incluye

- Inicio de sesión con dos roles: conductor y mecánico
- Registro de motos y carros por conductor
- Panel por vehículo: kilometraje, próximo mantenimiento, SOAT, técnico-mecánica, impuesto y pico y placa
- Lista de mantenimientos con intervalo en km y en meses
- Registro de mantenimientos realizados por el taller
- Datos guardados en el teléfono



No incluye

- Notificaciones push: “avisar” es mostrarlo en el panel
- Chatbot, sugerencias de repuestos y comparación de componentes
- Consulta automática a fabricantes: los planes son datos fijos por modelo
- Gasolina, batería, presión de llantas (sensores)
- Gastos, compras o pagos
- Registro de cuentas nuevas, iOS, web y pico y placa fuera de Medellín

Quién usa la app



Conductor

- Registra sus vehículos y actualiza el kilometraje
- Ve el panel de cada uno: qué vence y cuándo
- Marca un mantenimiento como hecho

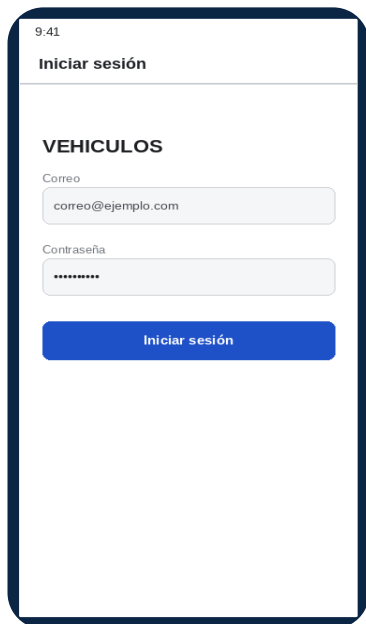


Mecánico / taller

- Ve los vehículos que atiende
- Registra el mantenimiento que hizo, con el kilometraje real
- Ese registro reinicia el contador del ítem (RN-05)

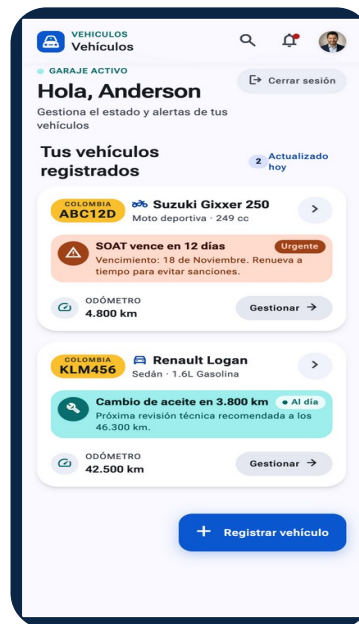
Login: correo y contraseña; el rol viene con la cuenta y decide a qué pantalla entra. **Stakeholders sin acceso:** Secretaría de Movilidad (pico y placa, impuesto), aseguradoras (SOAT), fabricantes (planes de mantenimiento).

Recorrido del conductor



P-01 Inicio de sesión

Correo y contraseña. El rol decide la pantalla siguiente.



P-02 Mis vehículos

Una tarjeta por vehículo con la alerta más próxima. Botón + y Cerrar sesión.



P-03 Panel del vehículo

Kilometraje, próximo mantenimiento, documentos y pico y placa. Botón Ver mantenimientos.

Tocar un vehículo en P-02 abre su panel: ese es el paso de lista a detalle.

Registrar, mantenimientos y taller

9:41

Registrar vehículo

Tipo de vehículo

☒ Moto ☐ Carro

Marca

Suzuki

Modelo

Gioxer 250

Año

2023

Placa

ABC12D

Ciudad

Medellin

Kilometraje actual

66000

Guardar

P-04 Registrar vehículo

Tipo, marca, modelo, año, placa, ciudad y kilometraje. Guardar vuelve a la lista.

Imagen del mockup
(docs/mockup)

P-05 Mantenimientos

Cada ítem con su intervalo ("cada 3.000 km o 3 meses"), cuánto falta, estado en color y botón Hecho.

Imagen del mockup
(docs/mockup)

P-06 Taller

Vehículos atendidos y formulario para registrar un mantenimiento: ítem, kilometraje y fecha.

P-06 es la pantalla del segundo rol: el mecánico entra directo aquí desde el login.

Demo en vivo



Recorrido completo del mapa de navegación, en el emulador

- | | | |
|---|----------------------|--|
| 1 | Login como conductor | conductor@correo.com / 1234 → Mis vehículos saluda con el correo |
| 2 | Tocar ABC12D | Panel de la Gixxer: 4.800 km, aceite, SOAT 06/10/2026 |
| 3 | Ver mantenimientos | Los ítems con su estado; el aceite ya vencido por meses |
| 4 | Atrás, atrás, + | Registrar vehículo, Guardar vuelve a la lista |
| 5 | Cerrar sesión | pop hasta el login, que sigue vivo debajo |
| 6 | Login como mecánico | taller@correo.com / 1234 → Taller |

Requerimientos principales

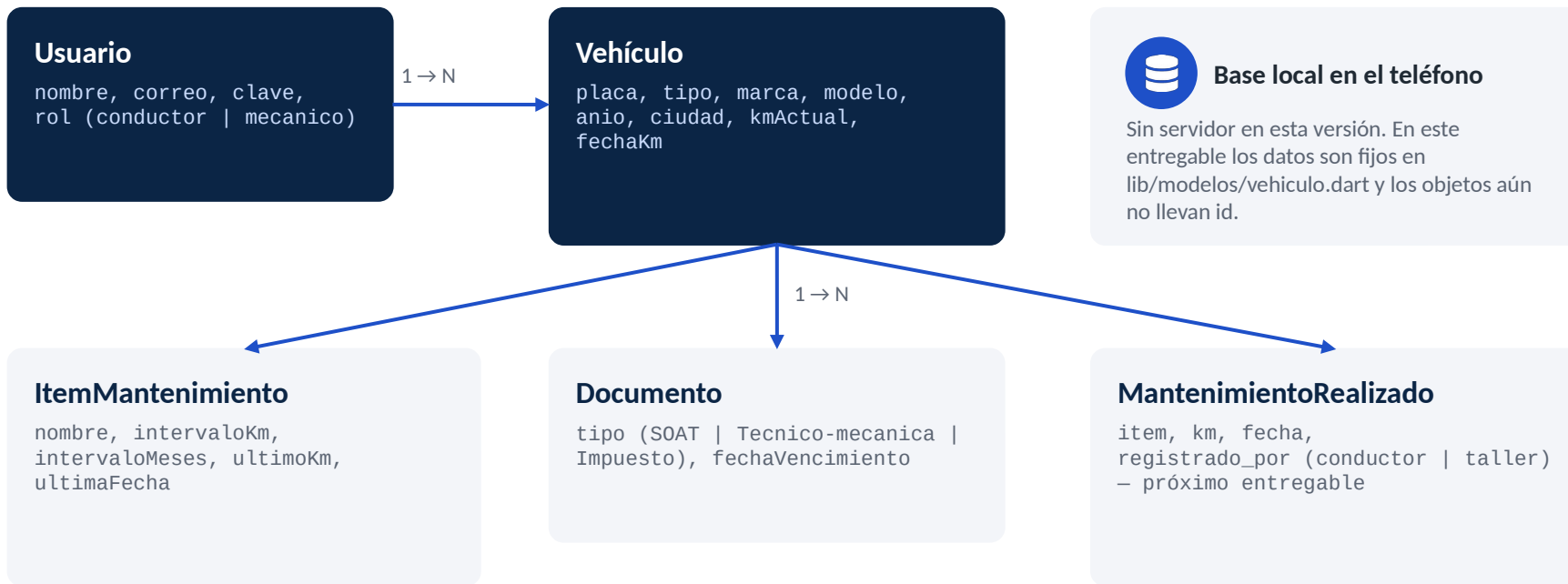
Funcionales

- RF-01** Iniciar sesión y llevar al usuario a la pantalla de su rol
- RF-03** Lista de vehículos del conductor con la alerta más próxima
- RF-04** Panel del vehículo tocado: km, mantenimiento, documentos, pico y placa
- RF-07** Calcular el vencimiento por km y por meses; mostrar el que llegue primero
- RF-08** Marcar un mantenimiento como hecho con kilometraje y fecha
- RF-12** Conservar los datos entre una apertura y la siguiente

No funcionales

- RNF-01** Sin conexión: todo se ve sin internet
- RNF-02** Abre y muestra la lista en < 3 s en gama media
- RNF-03** Táctil $\geq 48 \times 48$ px; la alerta se lee sin abrir el panel
- RNF-05** Android 8.0+, pantallas de 5 a 6,7", vertical
- RNF-06** La contraseña nunca se guarda en texto plano

Modelo de datos



Un mantenimiento realizado actualiza ultimoKm y ultimaFecha de su ítem: así se reinicia el contador (RN-05).

Mapa de navegación



Todas las flechas son `Navigator.push`; se vuelve con `Navigator.pop`. **Pila del conductor:** P-01 → P-02 → P-03 → P-05. Atrás desapila una a una; Cerrar sesión en P-02 deja solo P-01, que siguió vivo debajo.

Cómo viaja el dato de la lista al detalle

```
// lib/pantallas/mis_vehiculos.dart
itemBuilder: (context, index) {
  final vehiculo = vehiculos[index]; // el de ESTA fila
  return TarjetaVehiculo(
    vehiculo: vehiculo,
    onTap: () {
      Navigator.push(
        context,
        MaterialPageRoute(
          builder: (context) =>
            PanelVehiculo(vehiculo: vehiculo),
        ),
      );
    },
  );
},
);
```

```
// lib/pantallas/panel_vehiculo.dart
class PanelVehiculo extends
StatelessWidget {
  const PanelVehiculo({
    super.key,
    required this.vehiculo,
  });

  final Vehiculo vehiculo;

  // build() lee vehiculo.placa,
  // vehiculo.kmActual,
  vehiculo.items...
}
```

Por qué llega el vehículo tocado y no otro: cada fila crea su propia variable vehiculo y su propio onTap, que la recuerda (closure). Se pasa por constructor con required: si falta, no compila.

El login decide la ruta por rol; se vuelve con pop

```
// lib/pantallas/login.dart
Usuario? encontrado;
for (final Usuario usuario in usuarios) {
  if (usuario.correo == _correo.text.trim() &&
      usuario.clave == _clave.text) {
    encontrado = usuario;
    break;
  }
}
if (encontrado == null) { /* Snackbar */ return; }

final Usuario usuario = encontrado;
if (usuario.rol == 'conductor') {
  Navigator.push(context, MaterialPageRoute(
    builder: (context) => MisVehiculos(correo: usuario.correo)));
} else {
  Navigator.push(context, MaterialPageRoute(
    builder: (context) => Taller(mecanico: usuario)));
}
```

```
// Mis vehículos: Cerrar sesión
IconButton(
  onPressed: () =>
    Navigator.pop(context),
  child: const Text('Cerrar sesión'),
)
```

```
// Sin flecha automática en P-02:
automaticallyImplyLeading: false
```

Pila

Antes del pop: P-01 → P-02

Después: P-01

pop no crea nada: descarta la de encima y deja ver la que ya existía. Por eso el login vuelve con el correo escrito.

Repositorio y decisiones

```
README.md      cómo ejecutar, cuentas, capturas
docs/definicion.md 14 secciones, referencias, IA
docs/mockup/    p01-login.png ... p06-taller.png
docs/presentacion/ estas diapositivas
lib/main.dart   home: PantallaLogin()
lib/modelos/    vehiculo.dart (clases + datos fijos)
lib/pantallas/  login, mis_vehiculos, panel_vehiculo,
                registrar_vehiculo, mantenimientos, taller
lib/widgets/    tarjeta_vehiculo.dart
```



github.com/kuryx/VEHICULOS

Rama por tarea y Pull Request para todo lo que entra a main. Commits propios de los tres, en documentación y en código.

Decisiones

- Kilómetros o meses, lo que llegue primero (RN-01): así sirve para la moto de diario y la de fin de semana.
- Documentos vencen solo por fecha; mantenimientos por km o meses.
- Rol de mecánico: el taller reinicia el contador con el km real.
- Los datos viajan por constructor con required, no por variables globales ni rutas con nombre.
- Solo lo visto en clase: sin paquetes, sin manejo de estado externo.
- Sin servidor: base local; en este entregable, datos fijos.

Uso de inteligencia artificial



Claude (chat y Claude Code), declarado en docs/definicion.md

Para qué

Ordenar el SRS de agosto en las 14 secciones; borrador de requerimientos, reglas y modelo; esqueleto de pantallas y navegación, pidiendo la explicación de cada parte.

Qué aceptamos

Documentos por fecha y mantenimientos por km o meses. El rol de mecánico. Las seis pantallas y el mapa.

Qué corregimos / descartamos

Un borrador con otra numeración se reacomodó. Se descartó la pantalla aparte de Documentación y las notificaciones push.

El problema, el tema, los roles, el caso real, los mockups, las referencias y el código de las pantallas los escribimos y decidimos nosotros.

Preguntas